Functions and Getting Help

Course: Kaggle Learn · Instructor: Colin Morris · Lesson 2 of 7

Objective: Learn how to get help with Python functions, define your own functions, return values, use default arguments, write docstrings, and pass functions to other functions.

1. Getting help with `help()`

Python has many built-in functions, and you don't need to memorize all of them. The `help()` function provides documentation explaining what a function does and how to use it.

```
help(round)
```

Output:

```
round(number, ndigits=None)
    Round a number to a given precision in decimal digits.
```

The output contains two important things:

- Function signature: `round(number, ndigits=None)` tells us the function's parameters.
- Description: Explains the purpose of the function.

Here, `number` is the number to round, and `ndigits` is an optional parameter specifying decimal precision.

Correct vs. incorrect usage

```
help(round)       # Correct
help(round(-2.01))  # Incorrect for looking up round's documentation
```

Why?

- `help(round)` passes the function itself to `help()`.
- `help(round(-2.01))` first executes `round(-2.01)`, then asks for help about the resulting value.

Remember: To look up a function, pass its name without calling it.

Discovering optional arguments

```
help(print)
```

The signature includes:

```
print(value, ..., sep=' ', end='\n', file=None, flush=False)
```

Important arguments:

Argument	Purpose	Example
`sep`	Separates multiple values	`sep=" - "`
`end`	Specifies what to append at the end	`end="!"`
`file`	Specifies the output destination	Used for streams or files
`flush`	Controls immediate flushing of output	`flush=True`

Example:

```
print(1, 2, 3, sep=" < ")
```

Output:

```
1 < 2 < 3
```

Without `sep`, Python uses a space by default.

```
print("Hello", "World")
# Hello World
```

You can also change the ending:

```
print("Hello", end="!")
print("World")
```

Output:

```
Hello!World
```

2. Defining your own functions

Built-in functions are useful, but programming often requires custom behavior. Python allows you to create your own functions using the `def` keyword.

Basic syntax

```
def function_name(parameters):    # Function body
    return result
```

A function definition has three main parts:

1. `def` — keyword used to define a function.
2. Parameters — inputs the function can receive.
3. Function body — indented code that executes when the function is called.

Example: Finding the smallest difference

```
def least_difference(a, b, c):
    diff1 = abs(a - b)
    diff2 = abs(b - c)
    diff3 = abs(a - c)
    return min(diff1, diff2, diff3)
```

This function calculates the absolute difference between every pair of numbers, then returns the smallest difference.

Let's break it down:

- `abs(a - b)` calculates the non-negative difference between `a` and `b`.
- `min(diff1, diff2, diff3)` selects the smallest of the three differences.
- `return` sends that result back to the code that called the function.

Example:

```
print(least_difference(1, 10, 100))
print(least_difference(1, 10, 10))
print(least_difference(5, 6, 7))
```

Output:

```
9
0
1
```

Calling a function

Defining a function does not execute its body. You must call it.

```
def greet():
    print("Hello!")
greet()
```

Output:

```
Hello!
```

Key distinction: Defining a function creates reusable instructions; calling a function executes those instructions.

3. The `return` statement

The `return` keyword ends the function immediately and sends a value back to the caller.

```
def square(number):
    return number * numberresult = square(5)
print(result)
```

Output:

```
25
```

Here, `square(5)` returns `25`, which gets stored in `result`.

`return` vs. `print()`

These are not interchangeable.

```
def add_print(a, b):
    print(a + b)
def add_return(a, b):
    return a + b
```

Using them:

```
x = add_print(3, 4)
y = add_return(3, 4)
print(x)  # None
```

```
print(y)  # 7
```

`add_print()` displays `7` but doesn't return it. `add_return()` gives the value `7` back so that other code can use it.

`print()`	`return`
Displays information	Sends a value back to the caller
Does not return the displayed value	Returns the specified value
Useful for output and debugging	Useful for calculations and reusable logic

What happens without `return`?

```
def add(a, b):
    result = a + bx = add(3, 4)
print(x)
```

Output:

```
None
```

Python automatically returns `None` when a function finishes without an explicit return value.

Important: `None` represents the absence of a meaningful returned value. It is not the same as `0` or `"None"`.

Functions with side effects

Some functions are useful even without returning a value.

For example:

```
print("Hello")
```

The `print()` function displays text on the screen. This is a side effect: an action performed by a function beyond returning a value.

Other examples include writing to a file or modifying external state.

4. Docstrings

A docstring is a string placed immediately after a function definition's header, inside the function body. It describes what the function does.

def least_difference(a, b, c): """Return the smallest absolute difference between any pair of three numbers.""" diff1 = abs(a - b) diff2 = abs(b - c) diff3 = abs(a - c) return min(diff1, diff2, diff3)

Docstrings are usually written using triple quotes (`""" ... """`) and can span multiple lines.

Why use docstrings?

- Explain the function's purpose.
- Document its inputs and return value.
- Make code easier for other developers to understand.
- Allow `help()` to display your documentation.

help(least_difference)

Python displays the function signature and its docstring.

Including examples in docstrings

def square(number): """Return the square of a number. >>> square(4) 16 >>> square(-3) 9 """ return number * number

The `>>>` examples illustrate expected usage and output. Python does not automatically execute these examples merely because they appear in a docstring.

Best practice: Write a clear docstring for functions that you intend to reuse or maintain.

5. Default arguments

A function can have parameters with default values. If the caller omits an argument, Python uses its default.

```
def greet(who="Colin"):
    print("Hello,", who)
```

Calling the function in different ways:

```
greet()
greet("world")
greet(who="Kaggle")
```

Output:

```
Hello, Colin
Hello, world
Hello, Kaggle
```

Explanation:

- `greet()` uses the default value `"Colin"`.
- `greet("world")` supplies the argument positionally.
- `greet(who="Kaggle")` supplies the argument by name.

Positional and keyword arguments

Positional argument:

```
greet("Aru")
```

Python matches the argument to a parameter by its position.

Keyword argument:

```
greet(who="Aru")
```

Python matches the argument by its parameter name.

Keyword arguments can make function calls clearer, especially when a function has many parameters.

Default arguments in practice

```
def calculate_price(price, quantity=1):
    return price * quantityprint(calculate_price(100))       # 100
    print(calculate_price(100, 3))    # 300
```

If `quantity` is omitted, its value is `1`.

6. Functions passed as arguments

Python allows functions to be passed to other functions just like values such as integers or strings.

Functions are objects in Python, so you can refer to a function by its name without calling it.

Example: A function that calls another function

```
def mult_by_five(x):
    return 5 * x
def call(fn, arg):
    return fn(arg)
print(call(mult_by_five, 1))
```

Output:

```
5
```

Here:

- `mult_by_five` is passed as the argument `fn`.
- `1` is passed as the argument `arg`.
- `fn(arg)` executes the supplied function with the supplied input.

Notice the difference:

```
call(mult_by_five, 1)     # Passes the function
call(mult_by_five(1), 1)  # Passes the result of calling it
```

The second expression does not work as intended because `mult_by_five(1)` evaluates to `5`, which is an integer rather than a callable function.

Calling a function twice

```
def squared_call(fn, arg):
    return fn(fn(arg))
```

Using `mult_by_five`:

```
print(squared_call(mult_by_five, 1))
```

Output:

```
25
```

Step by step:

1. `mult_by_five(1)` returns `5`.
2. `mult_by_five(5)` returns `25`.
3. The final value is returned.

Higher-order functions

A higher-order function is a function that accepts another function as an argument, returns a function, or both.

Examples include:

- Custom functions such as `call()`.
- Built-in functions such as `max()` when used with its `key` argument.
- Functions such as `map()` and `filter()`.

Higher-order functions help you write reusable, flexible code.

7. The `key` argument in `max()`

Normally, `max()` returns the largest value.

```
print(max(100, 51, 14))
```

Output:

```
100
```

However, `max()` also accepts a `key` argument. This lets you specify a function that determines how each value is evaluated for comparison.

Example: Largest remainder after division by 5

```
def mod_5(x):
    """Return the remainder when x is divided by 5."""
    return x % 5
```

Now compare:

```
print(max(100, 51, 14, key=mod_5))
```

Output:

```
14
```

Why?

Number	`number % 5`
100	0
51	1
14	4

The largest remainder is `4`, so `max()` returns the original number `14`.

Important: `key=mod_5` passes the function itself, not the result of calling it. Python uses the function to calculate comparison keys.

This pattern is also useful for selecting strings by length, sorting objects by a property, or finding the item with the highest score.

8. Trailing commas

Python permits a trailing comma after the final argument in many function calls.

```
print(
least_difference(1, 10, 100),
least_difference(1, 10, 10),
least_difference(5, 6, 7),
)
```

The trailing comma after the final function call is valid.

This can make multiline code easier to edit, especially when adding or removing arguments.

9. Common mistakes to remember

Mistake	Why it is wrong	Correct approach
`help(round(3.5))`	Gets help for the result, not the function	`help(round)`
Forgetting `return`	The function returns `None` by default	Return the desired value
Confusing `print` with `return` .	Displaying a value doesn't return it	Use `return` when other code needs the value
Calling a function when passing it	Passes the result instead of the function	Use `fn`, not `fn()`
Forgetting indentation	Python uses indentation to define code blocks	Indent the function body consistently
Omitting required arguments	The function may raise a `TypeError`	Supply the required arguments or . define defaults